

# Experiment # 04

## Implementation of Profiling in C++ using GPROF and Timer

### Objectives:

The objective of this lab is to profile the complex code using gprof and acquire the more accurate time using the timer header file.

### Software Used:

Ubuntu

### Theory:

#### Gprof:

Gprof is a performance analysis tool used to profile applications to determine where time is spent during program execution. Gprof is included with most Unix/Linux implementations, is simple to use, and can quickly show which parts of an application take the most time. It can also tell you which functions are being called more or less often than you expected.

#### How to use gprof

Using the gprof tool is not at all complex. You just need to do the following on a high-level:

- Have profiling enabled while compiling the code
- Execute the program code to produce the profiling data
- Run the gprof tool on the profiling data file (generated in the step above).

The last step above produces an analysis file which is in human readable form. This file contains a couple of tables (flat profile and call graph) in addition to some other information. While flat profile gives an overview of the timing information of the functions like time consumption for the execution of a particular function, how many times it was called etc. On the other hand, call graph focuses on each function like the functions through which a particular function was called, what all functions were called from within this particular function etc So this way one can get idea of the execution time spent in the sub-routines too.

#### Timer:

Although the prof/gprof method provides more detailed information than the first few methods presented, it is often necessary to measure execution time with finer granularity than a function. Suppose prof was used and it shows that 90% of the time is spent in one subroutine. That subroutine becomes the primary target for optimization. But if the routine includes several loops, the next step is then to identify the most time-consuming parts within that subroutine. A possible approach is to use the timer() function, as provided by many operating systems, including UNIX. In this case, however, the program must be instrumented such that the timer is read at the beginning and end of the code segment being measured. Instrumenting the code means adding lines of code explicitly to

perform the timing measurements. Such lines of code are temporary and are removed once the desired data has been collected. This method is useful for fine-grain measurements, such as a code segment or loop, but it is not as convenient as `prof/gprof` to obtain measurements of multiple functions or processes at once. Here is an example of a program that uses `timer()`.

### Procedure:

First open MobaXterm or login to your system.



Enter Remote host IP address and your Host username. Click OK

Or

Insert your Password and you are logged in.



After successful login. Clear the screen by typing “clear” in the command window.

Right now, you are at your home directory. Which is “home/username”

Now create a new directory with the “mkdir” for this lab.

```
fasih@fasih-VirtualBox:~/PDClab04$ clear
```

It will create the new directory named “lab04”

Now change your directory to lab04 by using “cd” command

```
fasih@fasih-VirtualBox:~$ mkdir PDClab04
fasih@fasih-VirtualBox:~$ cd PDClab04
```

Here you have to Paste the example code given to you.

To execute this code All you have to do is write a command “make gprof”

```
fasih@fasih-VirtualBox:~/PDClab04$ gedit test.cpp
fasih@fasih-VirtualBox:~/PDClab04$ make gprof
```

test.cpp

```
1  #include <iostream>
2  #include "Timer.h"
3  #include <stdlib.h>
4  #include <stdio.h>
5  #include <math.h>
6
7  using namespace std;
8
9  int frequency_of_primes (int size) {
10     int i,j;
11     int n = size;
12     int freq=n-1;
13     for (i=2; i<=n; ++i) for (j=sqrt(i);j>1;--j) if (i%j==0) {--freq; break;}
14     return freq;
15 }
16
17 int frequency_of_primes2 (int size) {
18     int i,j;
19     int n = size;
20     int freq=n-1;
21     for (i=2; i<=n; ++i) for (j=sqrt(i);j>1;--j) if (i%j==0) {--freq; break;}
22     return freq;
23 }
24
25
26 int main()
27 {
28     int i=0, g, g2;
29     int size = 1e6;
30     Timer Freq1_Time("Function1 Time"), Freq2_Time("Function2 Time");
31
32     cout << "C++ source file" << endl;
33
34     for(i=0;i<size;i++); //some dummy delay
35
36     Freq1_Time.Start();
```

```

37     g = frequency_of_primes (size);
38     Freq1_Time.Stop();
39
40
41     Freq2_Time.Start();
42     g2 = frequency_of_primes2 (size);
43     Freq2_Time.Stop();
44
45     for(i=0;i<size;i++); //some dummy delay
46
47     cout << "Freq1: " << g << endl;
48     cout << "Freq2: " << g2 << endl;
49
50     Freq1_Time.Print();
51     Freq2_Time.Print();
52
53     return 0;
54 }

```

It will show you the execution time taking by each function. And ignore above warnings.

Now refresh your current directory.

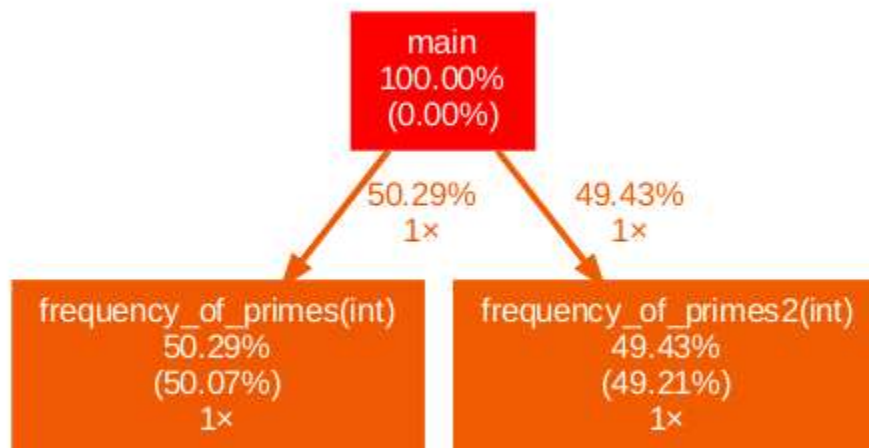
```

fasih@fasih-VirtualBox:~/PDClab04$ gedit test.cpp
fasih@fasih-VirtualBox:~/PDClab04$ make gprof

```

Now You will be able to see a pdf file named “profile-gprof.pdf”.

Open the file.



Now compare the results.

```
fasih@fasih-VirtualBox:~/PDClab04$ make gprof
rm -f test test.o *~ *.P perf.data gmon.out profile-gprof.* profile-perf.*
g++ -O1 -g -pg -fno-inline -fno-omit-frame-pointer -Wall -Wfatal-errors -I. -c
test.cpp -o test.o
cp: cannot stat 'test.d': No such file or directory
/bin/sh: 2: cannot open test.d: No such file
g++ test.o -o test -L. -lm -pg
./test
C++ source file
Freq1: 78498
Freq2: 78498
Function1 Time : 3750.42 msec
Function2 Time : 3556.61 msec
gprof -b test > profile-gprof.txt
bash ./gprof2pdf.sh profile-gprof.txt
fasih@fasih-VirtualBox:~/PDClab04$
```

So, we can see that by including Timer library we determined the exact time that one function or chunk of code took. While by Gprof method we just observe the percentage of each function.

---

#### Lab Tasks:

1. Run the above mention example code and compare the results.
2. Run the last lab code of timer and include Gprof in it.
3. Write a C++ program to find the factorial within a main function. Also write two functions to find:
  - a. Total number of prime numbers in the given size
  - b. Cube of each number & add resultant numbers.

Take a size of one million for each function.

Find time taking by each function using Timer library as well as using Gprof.

---